

Distributed Job Scheduler

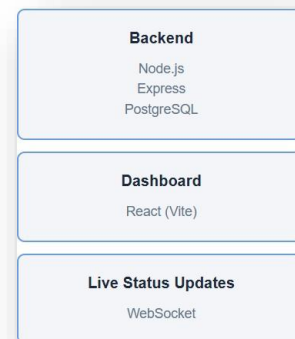
Codity.AI Tech Role Assignment — Submission Repository:

<https://github.com/SiD4422/Codity.ai>

Live Demo :- coditai.netlify.app

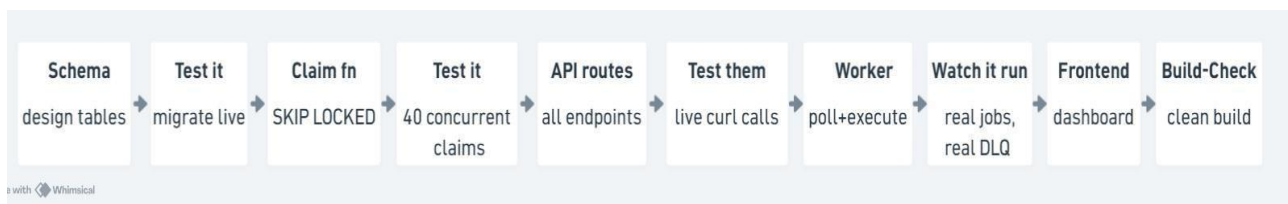
1. Overview

A platform for scheduling and reliably executing background jobs across multiple concurrent workers. Supports five job types (immediate, delayed, scheduled, recurring/cron, batch), atomic job claiming with zero doubleexecution, configurable retry policies with exponential backoff, a Dead Letter Queue for permanently failed jobs, and a live dashboard for monitoring queues and workers. Stack: Node.js + Express + PostgreSQL for the backend and worker processes; React (Vite) for the dashboard; native WebSocket for live status updates.



Development approach

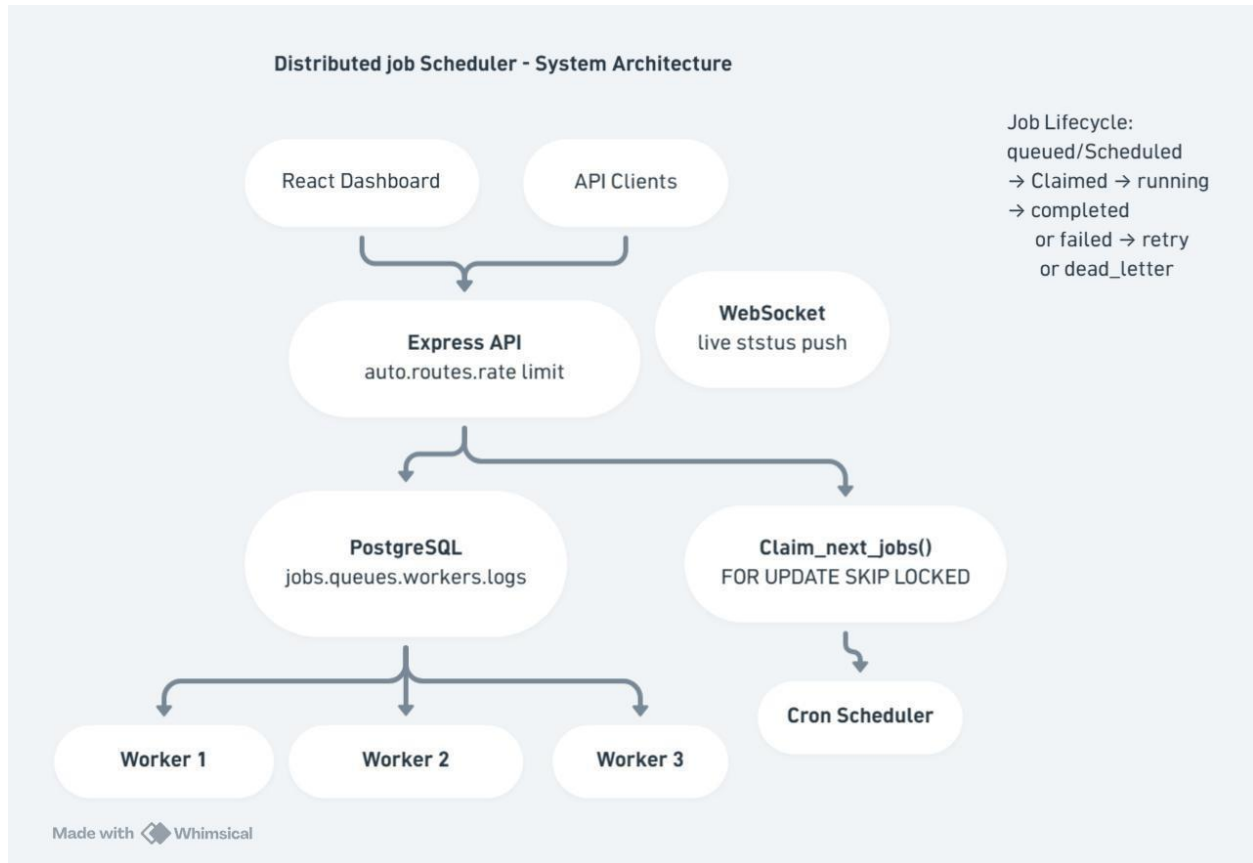
Built iteratively (Iterative/Incremental model), not front-loaded like a strict V-model: each piece was built, then immediately verified against a live local Postgres instance and a live worker process, before moving to the next piece. Documentation was written last, describing behavior already proven working rather than behavior assumed to work. This is why Section 5 below can report an actual bug found and fixed during development, not just a design that was never run.



2.

System Architecture

Postgres is the single source of truth — not just for data, but for coordination. Rather than adding a separate message broker (Redis/RabbitMQ), safe concurrent job claiming is implemented with a single atomic SQL function using `SELECT ... FOR UPDATE SKIP LOCKED`. This gives broker-grade ‘exactly one worker gets this job’ guarantees without a second system to operate and keep consistent with the database.

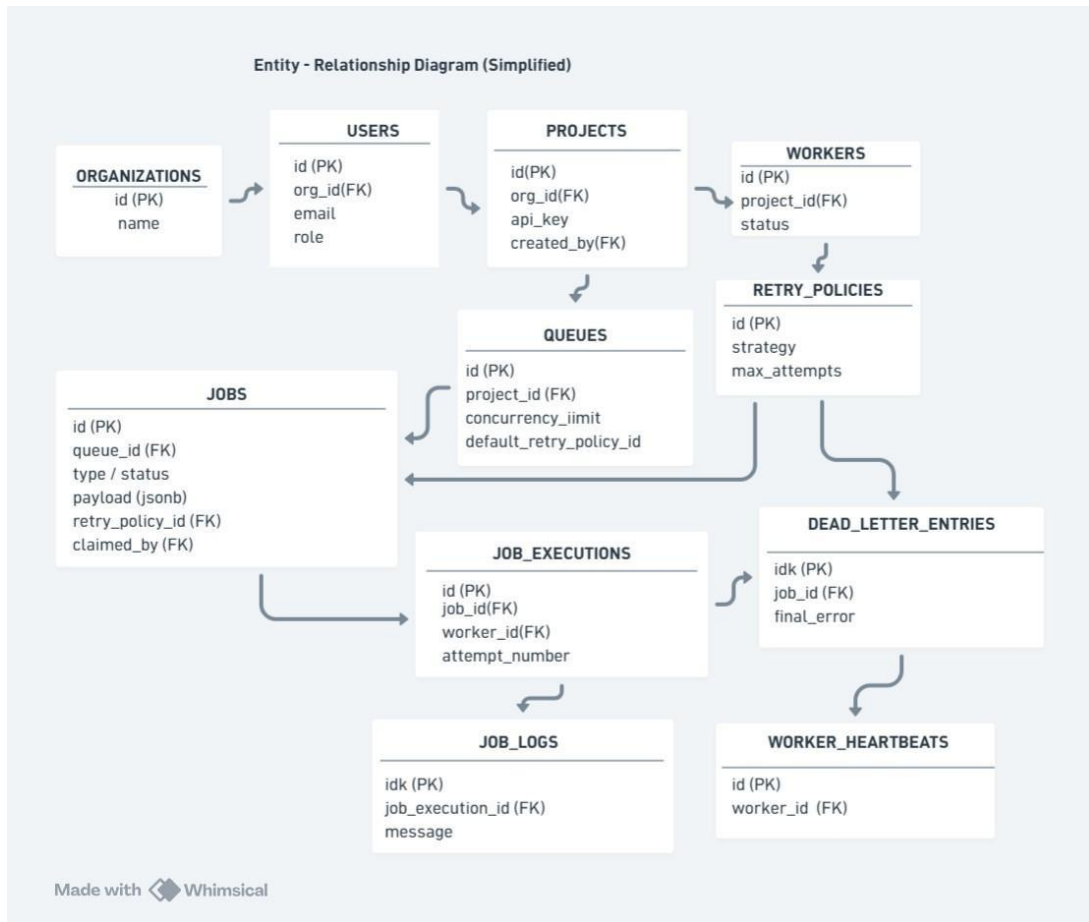


Job lifecycle: queued/scheduled → claimed → running → completed, with a failure branch that returns to scheduled (retry with backoff) until attempts are exhausted, at which point the job moves to dead_letter.

Database Design

Fully normalized (3NF) relational schema, with two deliberate denormalizations explained in the design notes below. Cascading deletes flow from organization → project → queue → job → execution → log, so nothing is ever orphaned.

3.



Key indexing decisions

- `idx_jobs_claim_lookup` — partial index on `(queue_id, run_at)` WHERE status IN ('queued','scheduled'). This is the hot path every worker poll hits; keeping it partial means completed/failed rows (the majority over time) never bloat it.
- `idx_jobs_idempotency` — partial unique index enforcing dedup only for jobs that opt in via an idempotency key.
- `idx_heartbeats_worker_time` — serves 'most recent heartbeats for worker X' without a sort at query time, on the highest-volume table in the schema.

Why `job_executions` is separate from `jobs`

A job can fail and retry multiple times; each attempt needs its own start/end time, duration, and error without overwriting the previous attempt. `jobs` stays small (one row per logical job); full attempt history lives in `job_executions`, with `job_logs` hanging off each execution rather than the job as a whole.

The Atomic Claim Function

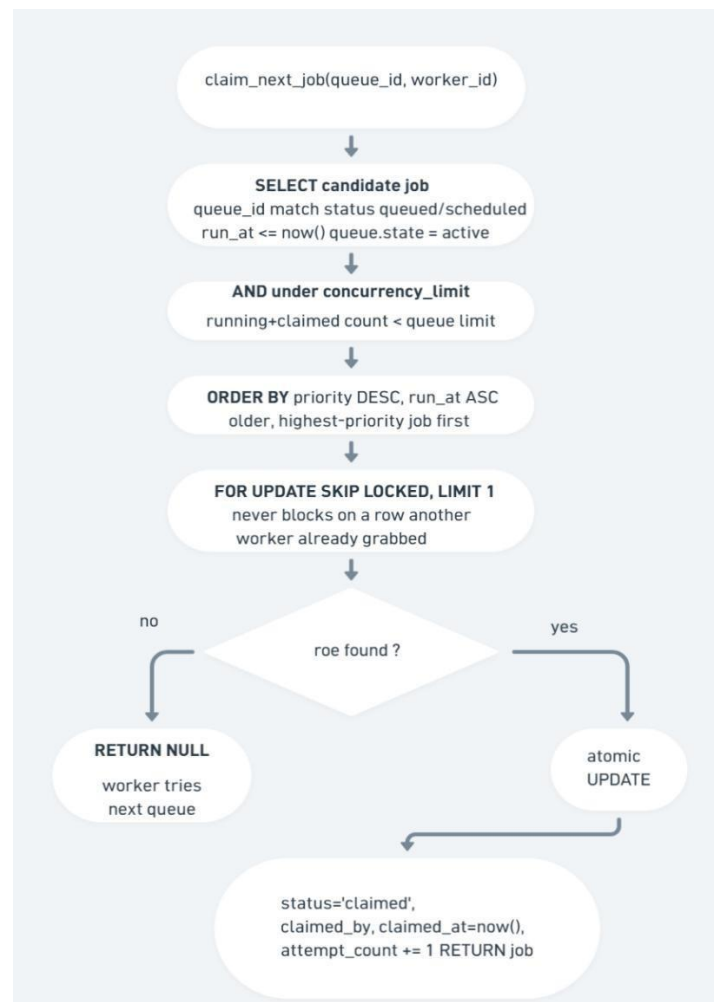
This is the core reliability guarantee of the whole system: many worker processes poll the same queue concurrently, and exactly one of them may claim any given job.

4.

```
CREATE OR REPLACE FUNCTION claim_next_job(
  p_queue_id UUID, p_worker_id UUID
) RETURNS jobs AS $$
DECLARE v_job jobs;
BEGIN
  SELECT j.* INTO v_job FROM jobs j JOIN queues q ON q.id = j.queue_id
  WHERE j.queue_id = p_queue_id
    AND j.status IN ('queued','scheduled') AND j.run_at <= now()
    AND q.state = 'active'
    AND (SELECT count(*) FROM jobs r WHERE r.queue_id = p_queue_id
        AND r.status IN ('claimed','running')) < q.concurrency_limit
  ORDER BY j.priority DESC, j.run_at ASC
  FOR UPDATE OF j SKIP LOCKED LIMIT 1;

  IF v_job.id IS NULL THEN RETURN NULL; END IF;
  UPDATE jobs SET status='claimed', claimed_by=p_worker_id,
  claimed_at=now(), attempt_count=attempt_count+1
  WHERE id = v_job.id RETURNING * INTO v_job;
  RETURN v_job;
END; $$ LANGUAGE plpgsql;
```

SKIP LOCKED means a worker that would otherwise block on a row another transaction has already locked instead skips it and looks at the next candidate — so N workers polling in parallel never queue up waiting on each other.



Verification — this was tested, not just written

Every major piece was run against a live local PostgreSQL 16 instance and a live worker process, not just written and assumed correct. In the process, one real bug was found and fixed: the claim function's result was initially being

5.

read from the wrong shape returned by node-postgres (a composite-returning SQL function auto-expands its columns directly into the row, rather than nesting under a key) — jobs were being claimed at the database layer but the worker was never receiving them back. Caught by testing the query in isolation, not assumed.

Automated test results (node --test)

```
ok 1 - claim_next_job never double-assigns under concurrent callers
ok 2 - fixed strategy returns constant delay (jitter off) ok 3 -
linear strategy scales with attempt number ok 4 - exponential
strategy doubles each attempt ok 5 - delay is capped at max_delay_ms
ok 6 - jitter keeps delay within [0, computed] ok 7 - shouldRetry
respects max_attempts
ok 8 - reaper requeues a job whose worker went silent, when retries remain ok 9 -
reaper dead-letters a job whose worker went silent, when retries are exhausted ok
10 - reaper does not touch jobs whose worker is still alive
# tests 10
# pass 10
# fail 0
```

The concurrency test fires 40 simultaneous `claim_next_job()` calls (simulating 20 workers, 2 attempts each) against a queue holding exactly 20 jobs, and asserts every claimed job ID is unique — proving zero double-execution under real concurrent load, not just by code review. The three reaper tests simulate a worker that crashed ungracefully (heartbeat goes silent while it still holds a job) and confirm: the job is requeued when retries remain, dead-lettered when they don't, and — just as importantly — a job on a live, healthy worker is left completely alone.

End-to-end manual verification

- Registered a user, created an organization, project, and queue via the live API.
- Created one job of each of the 5 types (immediate, delayed, scheduled, recurring, batch) and confirmed correct status/run_at handling for each.
- Started a real worker process; watched it claim, run, and complete a noop job end-to-end.
- Ran a fail-always job through 5 attempts with exponential backoff, watched it land in the Dead Letter Queue, then manually retried it back to queued via the API.
- Started the cron scheduler process and watched it materialize a real job row from a recurring definition.
- Built and served the React dashboard; confirmed a clean production build with no errors.
- Validated the OpenAPI spec (docs/openapi.yaml) with swagger-cli — confirmed well-formed, not just plausible-looking YAML.

6.

API Summary

Base URL: <http://localhost:4000/api/v1> (versioned so future breaking changes don't disturb existing clients). Machine-readable spec: [docs/openapi.yaml](#), validated with `swagger-cli`.

Endpoint	Purpose
POST /auth/register, /auth/login	Create workspace / sign in, returns JWT
GET/POST /projects	Manage projects (each gets a worker API key)
GET/POST/PATCH /queues, /:id/pause, /:id/resume, /:id/stats	Queue config, control, and live stats
POST /jobs	Create a job of any of the 5 types
GET /jobs, /:id, /:id/logs	Paginated job explorer, detail, and structured logs
POST /jobs/:id/cancel, /:id/retry	Cancel a pending job / requeue from the DLQ
GET /workers, /:id/heartbeats	Live worker fleet status
GET /dlq	Dead Letter Queue listing
WS /ws	Live job-status-summary push every 3s

Full request/response documentation: see [docs/api.md](#) in the repository.

7. Design Decisions & Trade-offs

- Postgres SKIP LOCKED instead of a dedicated broker — one less stateful system to operate; proven sufficient at this scale via the concurrency test above.
- Cron materialization runs as a separate singleton process, not inside each worker — otherwise scaling workers from 1 to 10 would fire every recurring job 10x.
- Dashboard uses REST polling (3s) plus a WebSocket aggregate push, rather than full per-row real-time diffs — true change-data-capture is its own project; this keeps the UI responsive without over-building for a class assignment.
- Idempotency keys are opt-in per job, not mandatory — avoids forcing complexity onto jobs that don't need deduplication.
- Workers authenticate with a static per-project API key rather than a JWT — they're long-running machine processes, not human sessions; this matches how Sidekiq Pro, Inngest, and Temporal Cloud handle worker auth.
- Stale job recovery is heartbeat-based, not job-age-based — the reaper watches for jobs whose claiming worker's heartbeat has gone silent (crashed ungracefully), not simply 'running a while', so it never misfires on a legitimately long job on a healthy worker. Recovered jobs go through the same retry/DLQ path a normal failure would.
- API layer is stateless and scales horizontally trivially (JWT auth, no server-side session) — the one caveat is the WebSocket push, which today is per-instance rather than synchronized across a multi-instance deployment; noted as a natural next step, not hidden.

7.

Explicitly not built (bonus scope, called out rather than hidden)

- Full RBAC beyond an admin/member column stub
- Workflow dependencies (job B waits on job A) — schema has room via batch_id but doesn't encode a DAG
- Queue sharding / multi-region — a single Postgres instance is this design's honest bottleneck ceiling

8. Running It Locally

Full instructions are in the repository README. Summary:

```
createdb jobscheduler cd backend && npm install && npm run  
migrate && npm start PROJECT_ID=<id> npm run worker      # in  
another terminal node src/workers/scheduler.js          #  
optional, for cron jobs  
node src/workers/reaper.js                             # recovers jobs from crashed workers  
cd frontend && npm install && npm run dev
```